

FinPilot

Investment Portfolio & Goal Monitoring Platform

Architecture & design report

Item	Detail
Author	rogerantony-dev
Date	25 September 2026
Repository	github.com/rogerantony-dev/finpilot
Stack	React 19 + Vite · Fastify 5 + Zod · Kysely · PostgreSQL 17 · Docker Compose · GitHub Actions
Run it	<code>cp .env.example .env</code> (set <code>JWT_SECRET</code>), then <code>docker compose up --build</code> → <code>http://localhost:8088</code>
Data	Synthetic data only, supplied CSVs loaded unmodified

Companion documents in the repository: [README.md](#) (setup and usage), [DECISIONS.md](#) (decision log D1–D12 with alternatives), [docs/data-audit.md](#), [docs/sql-tasks.md](#) (queries and `EXPLAIN` walkthrough) and [docs/diagrams/](#) (diagram sources).

1. Executive overview

Problem. A wealth-service team needs one place to see a customer’s consolidated investment position (accounts, holdings, activity, risk profile and goals) and an operations administrator needs to load daily CSV data safely: validated, idempotent, with every rejected row explained.

Scope delivered. Sign-in with VIEWER and ADMIN roles; customer search; customer overview with portfolio value, gain/loss, data freshness, asset-allocation chart and account cards; positions; server-side filtered and paginated transactions; goals with create/edit and consistency flags; administrator CSV import with row-level reasons and import history. A versioned REST API (`/api/v1`) with OpenAPI docs, PostgreSQL schema via migrations, a one-command Docker stack and a CI pipeline that tests the code and the running stack.

Major design decisions (full log with rejected alternatives in [DECISIONS.md](#)):

- **TypeScript end-to-end** in a pnpm monorepo; request/response contracts are Zod schemas in [packages/shared](#), used for API validation, serialisation, the OpenAPI document and frontend types, so documentation cannot drift from code.
- **PostgreSQL does the data work:** constraints encode business rules; views hold the single definition of market value, P/L, allocation and goal status; the API never aggregates in application code and the browser never does valuation arithmetic.
- **Plain SQL migrations (dbmate) + Kysely** rather than an ORM, keeping constraints, views, partial indexes and `EXPLAIN` visible and reviewable.
- **One import pipeline** for the seed and the admin upload: stage → validate → merge in one transaction, idempotent by file SHA-256, rejects quarantined with reasons. The seed therefore proves the pipeline on the supplied data: **transactions.csv loads 4,550 rows and rejects the 4 planted anomalies; holdings de-duplicates 3 exact duplicates.**
- **Same-origin deployment behind nginx** with a JWT session in an `httpOnly`, `SameSite=Strict` cookie: no token in JavaScript, no CORS, API not exposed.

Evidence. 62 automated tests (53 API/integration against a real PostgreSQL, 9 web), a GitHub Actions pipeline with a Docker-stack smoke test, and a deliberately failing pull request (#1) showing the pipeline blocks broken code.

2. System context

Three people use the system: a **wealth/service user** (VIEWER) who looks up customers and maintains goals, an **operations administrator** (ADMIN) who imports daily files, and an **engineering reviewer** who inspects the API, database, logs and pipeline. Everything runs locally in Docker; the only external platform dependency is GitHub (repository, Actions CI, Dependabot). No external market data, identity provider or payment system is involved.

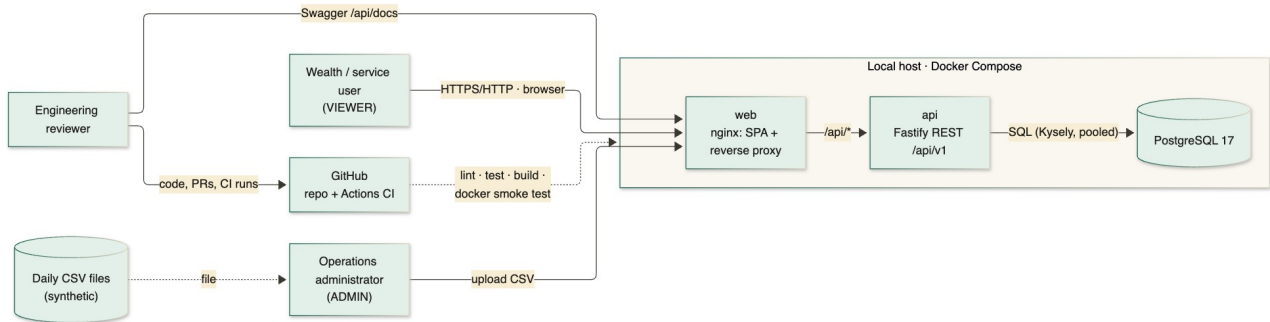


Figure 1 · System context

3. Component architecture

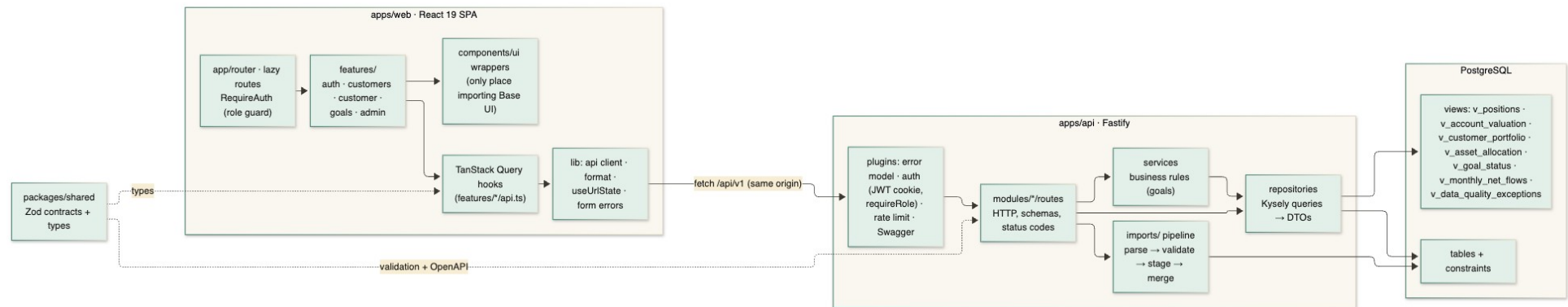


Figure 2 · Components and data-access boundary

Component	Responsibility
web · <code>app/router</code>	Routes; customer pages lazily loaded (chart library only on the overview); <code>RequireAuth</code> guards pages and the ADMIN-only import route.
web · <code>features/*</code>	Screens per area. Server state only through TanStack Query hooks in <code>features/*/api.ts</code> ; no <code>useEffect</code> in the app. Filters, sort and page live in the URL.
web · <code>components/ui</code>	Wrappers around Base UI primitives (Select, Dialog, Form/Field, Meter, Progress, Toast) plus local primitives (Table, Card, Badge). An ESLint rule forbids importing Base UI anywhere else.
shared · <code>packages/shared</code>	Zod contracts and inferred types for every request and response.
api · <code>plugins</code>	Error model, JWT-cookie auth and <code>requireRole</code> , login rate limit, request IDs, log redaction, Swagger.
api · <code>routes</code> → <code>services</code> → <code>repositories</code>	Routes handle HTTP only; services hold business rules (goal dates, over-funding); repositories are the only data-access boundary: parameterised Kysely queries mapping snake_case rows to camelCase DTOs.
api · <code>imports/</code>	Generic CSV pipeline driven by per-dataset definitions (columns, row schema, natural key, references).
PostgreSQL	Tables with constraints; reporting views used by repositories.

4. Data model (ERD)

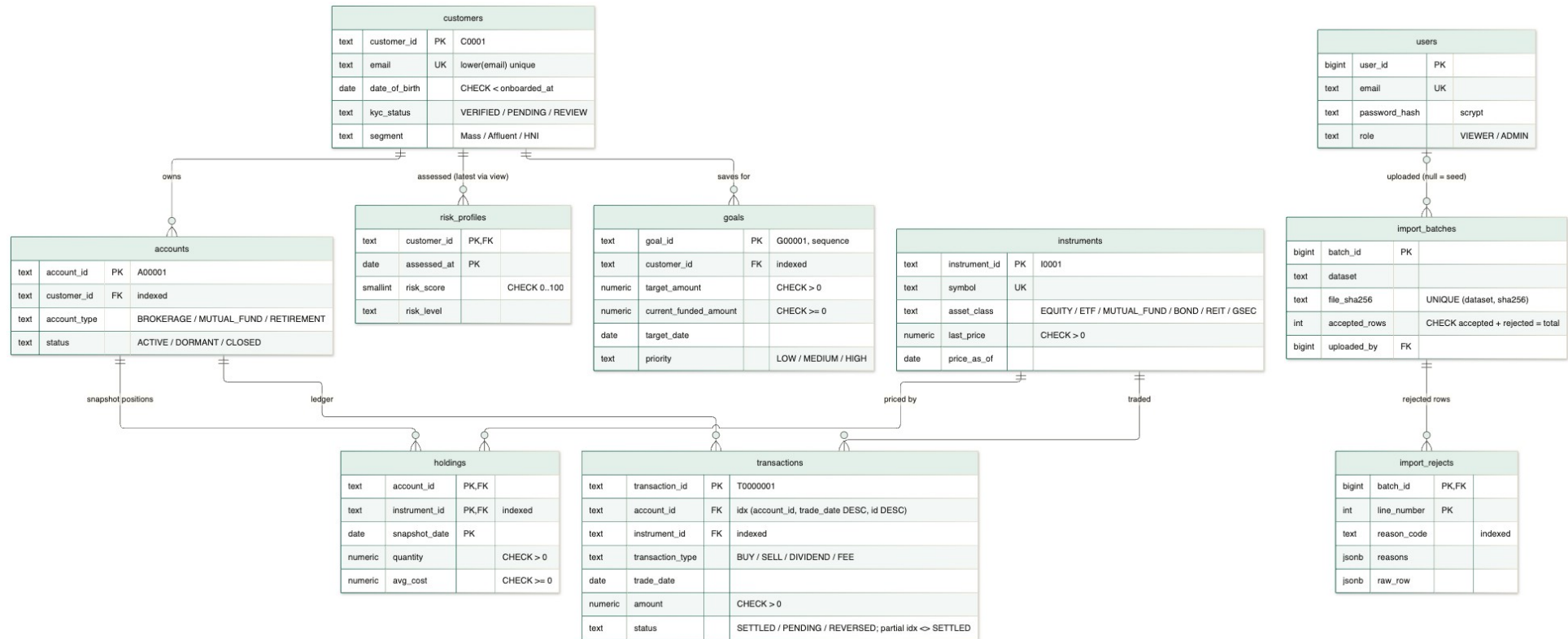


Figure 3 · Entity-relationship diagram (PK, FK, UK, key CHECK constraints and indexes)

Keys and types: natural keys from the data (C00001, A000001, T00000001) with format CHECKs; **NUMERIC** money/prices/quantities, never floats; **DATE** for dates; enums as **TEXT + CHECK**. Risk profiles keep history (latest via view); app-created goals take IDs from a sequence. **Constraints as last line of defence:** `amount > 0`, trade vs cash-event shape, `date_of_birth < onboarded_at`, sector only on equities, unique email, `accepted + rejected = total`, **UNIQUE** (dataset, file_sha256).

Indexes from query patterns only: `transactions(account_id, trade_date DESC, transaction_id DESC)` for the paginated ledger, a partial index on non-settled rows, and FK look-ups on accounts, goals, holdings, transactions. **Views:** `v_positions` (value = quantity × last price; P/L = rounded value – rounded cost, so totals add up), `v_account_valuation`, `v_customer_portfolio`, `v_asset_allocation`, `v_goal_status`, `v_monthly_net_flows`, `v_data_quality_exceptions`, `v_latest_risk_profile`.

5. API design

Method	Path (prefix /api/v1)	Purpose	Auth
GET	/health	Liveness + DB readiness (200 ok / 503 degraded), no secrets	public
POST	/auth/login · /auth/logout	Start (sets cookie) / end session	public
GET	/auth/me	Current user	session
GET	/customers	Search q (ID prefix, name, email, city), filters, sort, pagination	session
GET	/customers/{id}	Profile + latest risk profile (null if none)	session
GET	/customers/{id}/portfolio	Totals, accounts, allocation, positions, snapshot and price dates	session
GET	/customers/{id}/transactions	Filters: from/to, account, instrument, type, status; sort; pagination	session
GET · POST	/customers/{id}/goals	List with funded % and flags · create	session
PATCH	/goals/{goalId}	Edit changed fields	session
POST	/admin/imports/transactions	CSV import (text/csv body)	ADMIN
GET	/admin/imports · /admin/imports/{id}/rejects	Import history · rejected rows	ADMIN

- **Contracts and docs:** every route declares Zod schemas for params, query, body and responses; Swagger UI at `/api/docs` is generated from them, with real example responses for the portfolio and import endpoints.
- **Validation:** malformed input → `400 VALIDATION_ERROR` with `details[{field, message}]`; well-formed but against a business rule → `422 UNPROCESSABLE` (e.g. a goal date in the past).
- **Error model:** `{ error: { code, message, details?, requestId } }` for every failure; codes `UNAUTHENTICATED` 401, `FORBIDDEN` 403, `NOT_FOUND` 404, `RATE_LIMITED` 429, `INTERNAL_ERROR` 500. Unexpected errors are logged with stack trace and returned generically.
- **Pagination:** `page / pageSize` (≤ 100) with `totalItems` and `totalPages`; every sort has a unique tie-breaker so pages never overlap. Customer sub-resources join through `accounts.customer_id`, so another customer's `accountId` returns nothing.
- **Conventions:** JSON in camelCase; money and quantities as decimal strings; dates `YYYY-MM-DD`. **Versioning** by URL prefix; a breaking change would ship as `/api/v2` alongside `v1`.
- **Import semantics:** `201` new batch, `200 ALREADY_IMPORTED` for a file seen before (nothing changes), `422` for a wrong header; up to 200 rejects inline, the rest paginated.

6. Workflows

6.1 Login

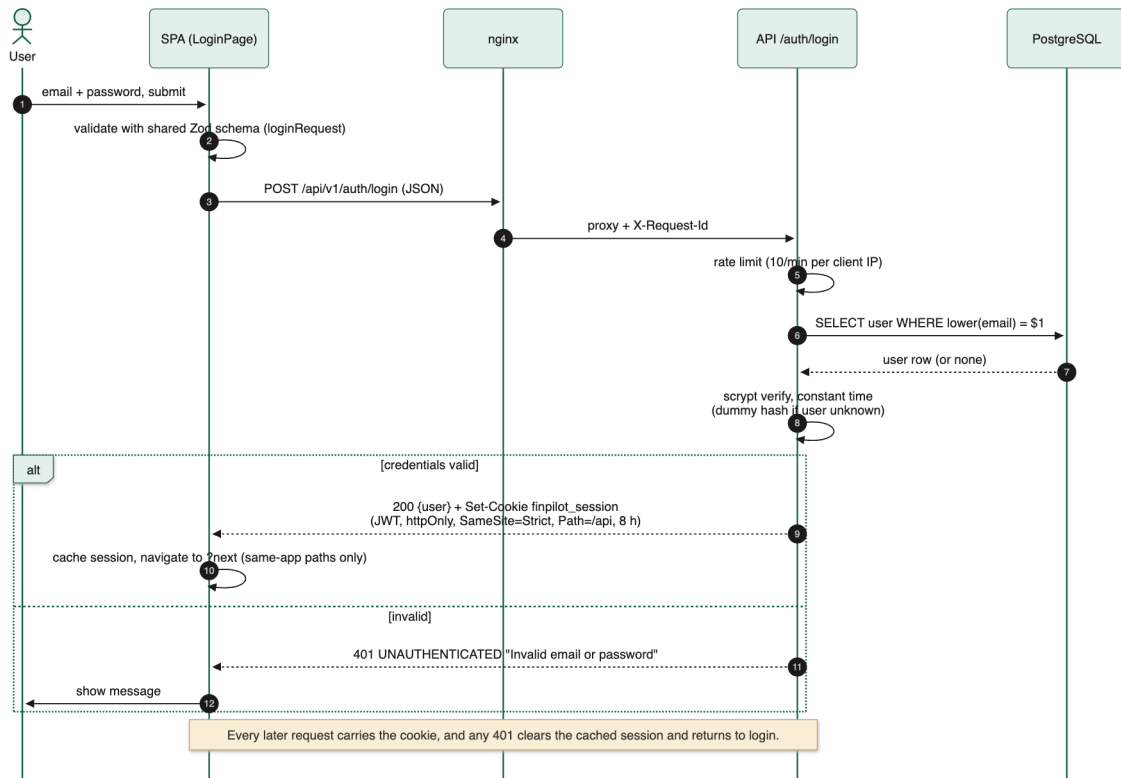


Figure 4 · Login and session establishment

6.2 Customer portfolio read

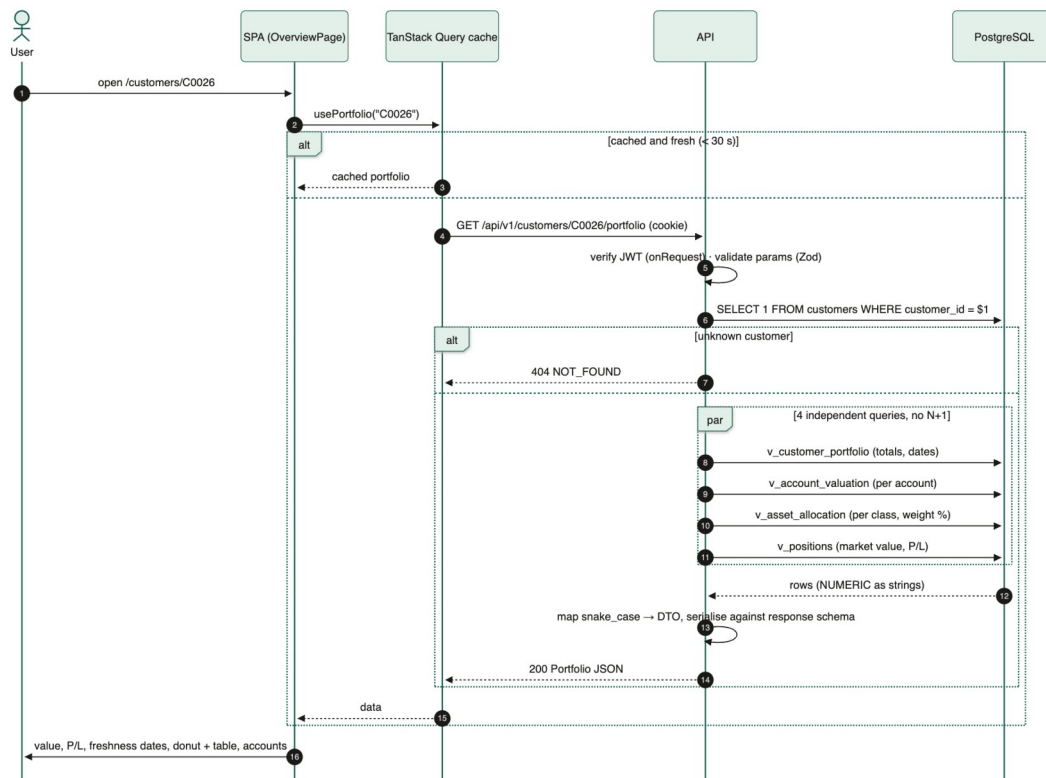


Figure 5 · Portfolio read: cache, auth, four view queries in parallel

6.3 Goal update

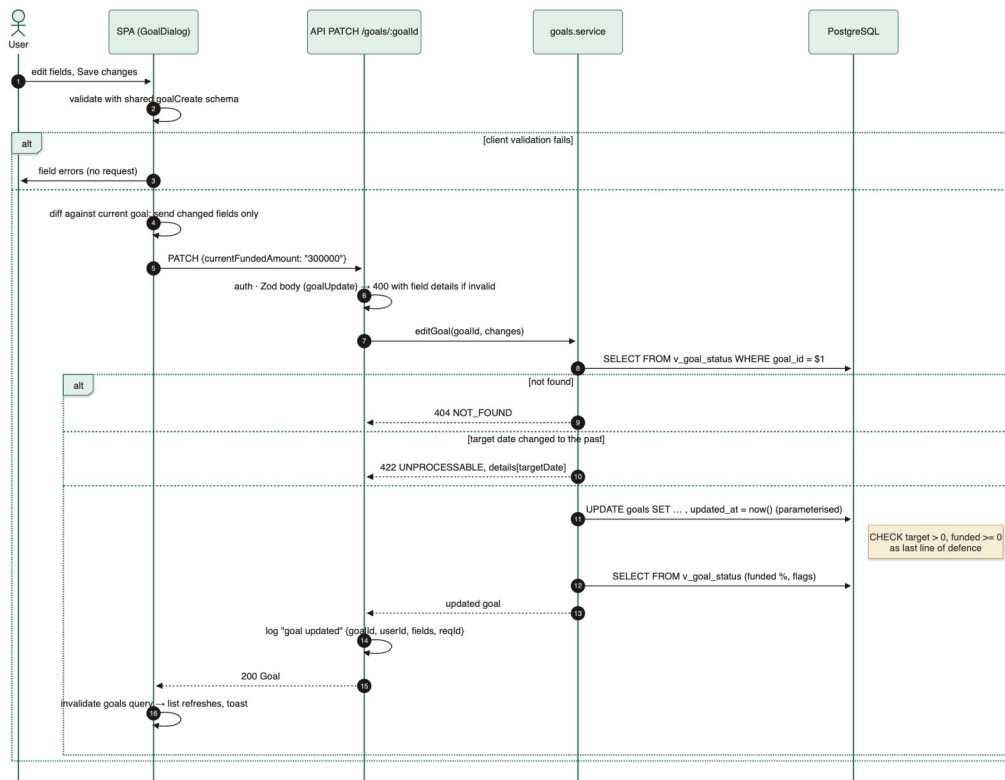


Figure 6 · Goal update: client validation, PATCH of changed fields, service rules, constraints

6.4 CSV import

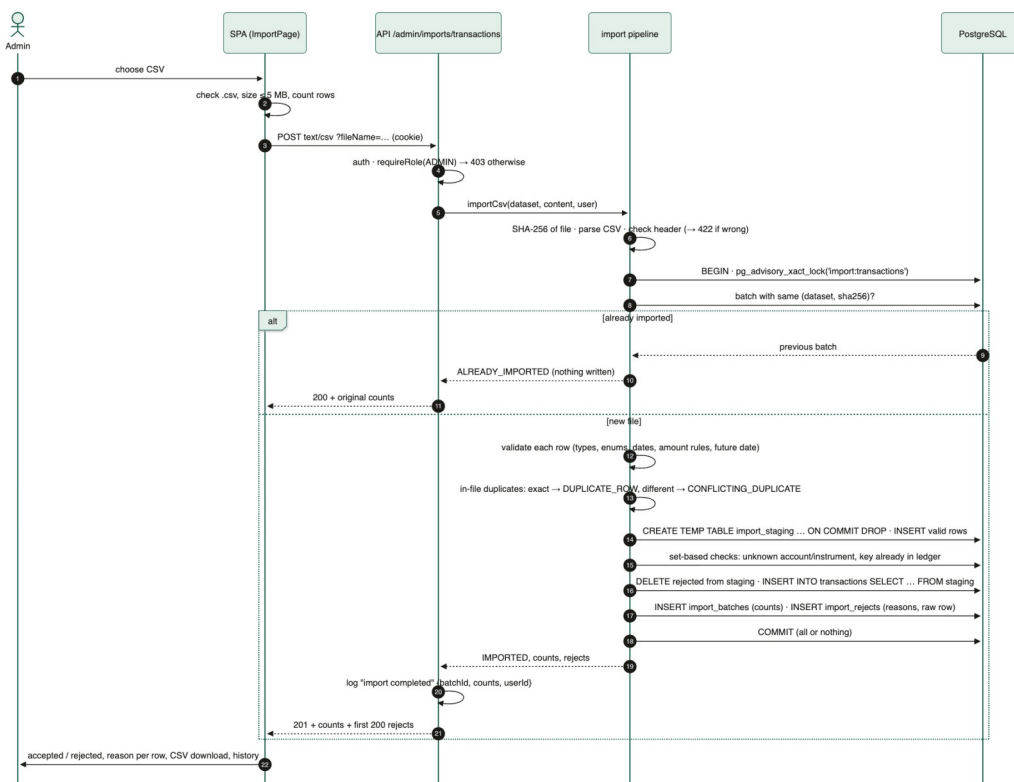


Figure 7 · CSV import: idempotency, staging, set-based checks, single-transaction merge

6.5 CI/CD

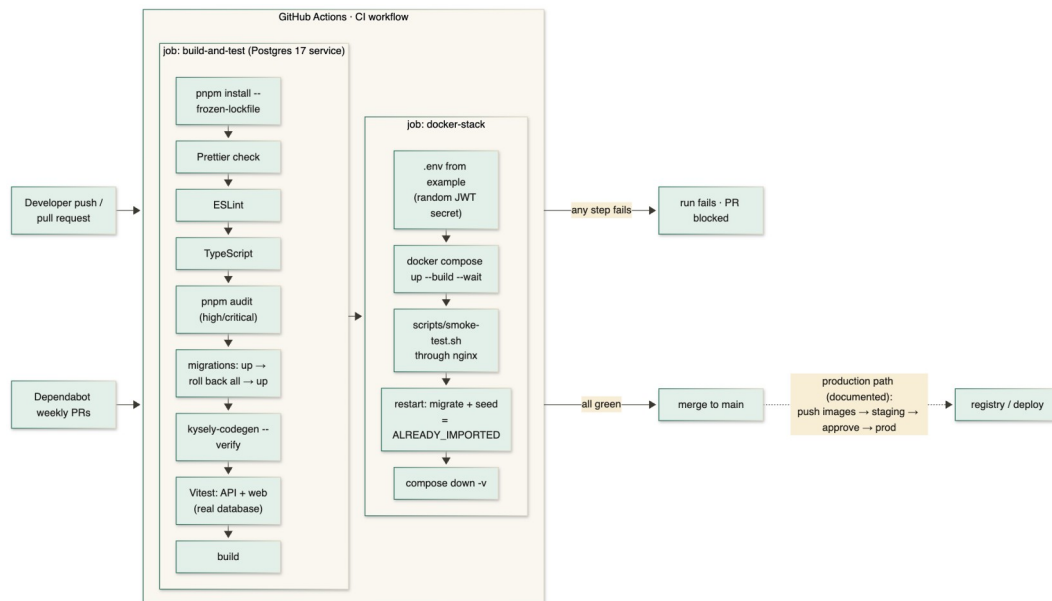


Figure 8 · CI/CD pipeline

7. Deployment topology

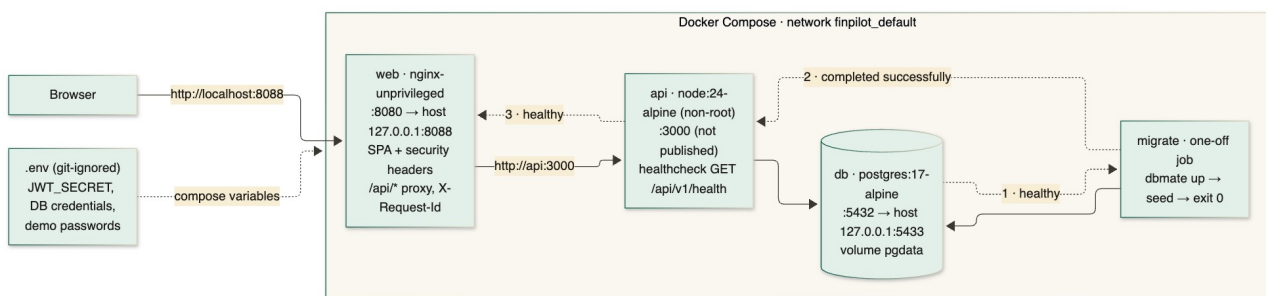


Figure 9 · Local deployment (Docker Compose)

Local. `docker compose up --build` builds two multi-stage, non-root images (API: compiled `dist/`, production dependencies, migrations, CSVs; web: static build on `nginx-unprivileged`). Start-up is ordered by health: PostgreSQL healthy → `migrate (dbmate up, idempotent seed, exit 0)` → API healthy → web. Only 127.0.0.1:8088 (web) and 127.0.0.1:5433 (PostgreSQL, for development) are published. Config comes from a git-ignored `.env`; Compose refuses to start without required secrets. **Environments:** development (`pnpm dev` with hot reload), CI (service container + Docker stack), and this Compose stack as production-like.

Local	Production mapping
nginx container	Managed load balancer / CDN terminating TLS (HSTS, <code>COOKIE_SECURE=true</code>); static assets on the CDN; domain via DNS + managed certificates.
api container	2+ replicas on a container platform (ECS, Cloud Run or Kubernetes) with health checks and rolling deploys.
migrate job	One-off release step before rollout; expand → migrate → contract migrations so old and new versions coexist.
PostgreSQL container	Managed PostgreSQL (multi-AZ, automated backups, point-in-time recovery) behind PgBouncer/platform pooling.
.env file	Secret manager with rotation; separate DB roles (DDL for migrations, DML-only for the app).
docker compose logs	Central JSON log aggregation, metrics and alerts (5xx rate, latency, failed imports).

8. Security

Area	Controls
Authentication	Seeded demo users; script password hashes with per-user salt, constant-time comparison, dummy hash for unknown emails (no user enumeration by timing or message); login rate limited to 10/min per client IP (proxy-aware).
Session	HS256 JWT (8 h) in an httpOnly, SameSite=Strict, Path=/api cookie, Secure when served over TLS; never readable by JavaScript or stored in localStorage. Trade-off: stateless, so role changes apply at expiry (logout clears the cookie immediately).
Authorisation	All <code>/api/v1</code> routes except health and login require a session; admin routes enforce <code>requireRole(ADMIN)</code> on the server; the UI hides admin features as a convenience only.
Input validation	Zod on every params/query/body; CSV rows validated field by field; body limits (1 MB JSON, 5 MB CSV); redirect targets after login restricted to same-app paths.
SQL injection	Only parameterised Kysely queries; dynamic identifiers in the import pipeline come from fixed dataset definitions, never from input; LIKE wildcards in search are escaped.
CORS / CSRF	Same origin through nginx; CORS allows only <code>WEB_ORIGIN</code> ; SameSite=Strict cookie; state-changing endpoints require JSON or text/csv bodies.
Browser hardening	CSP <code>default-src 'self', frame-ancestors 'none'</code> , X-Frame-Options DENY, nosniff, Referrer-Policy, Permissions-Policy; API responses <code>Cache-Control: no-store</code> ; pages restored from the back/forward cache are reloaded so data is not shown after sign-out.
Secrets	<code>.env</code> git-ignored, <code>.env.example</code> committed with placeholders; demo passwords from environment, not code; GitGuardian scan on pull requests; <code>pnpm audit</code> and Dependabot for dependencies.
Logs and PII	JSON logs redact Cookie, Authorization and Set-Cookie; no passwords or row contents logged; data is synthetic. Production: mask email/phone in logs, restrict DB access, encryption at rest by the managed database.

9. Reliability and operations

- **Health checks:** `/health` reports API status and database connectivity (200 ok / 503 degraded) and backs the container health checks that order start-up.
- **Logging and correlation:** structured JSON logs (pino); every request gets an ID, honoured from nginx's `X-Request-Id`, returned in the `x-request-id` header and included in every error body, so a user-visible error maps to one log trail. Business events are logged explicitly: login success/failure, forbidden access, goal created/updated (user, fields), import completed (batch, counts, user).
- **Timeouts:** database connect timeout 5 s and statement timeout 10 s per pooled connection; nginx proxy read timeout 60 s; the frontend retries network and 5xx failures once, never 4xx.
- **Failure behaviour:** imports are all-or-nothing per file (one transaction), serialised per dataset by an advisory lock, and idempotent, so a retry after a crash is safe. A failed migration stops the start-up chain and dbmate rolls that migration back (each runs in a transaction); CI proves every migration also rolls back cleanly. Graceful shutdown drains in-flight requests and closes the pool.
- **Monitoring (production):** alerts on 5xx rate and latency percentiles, failed or high-reject imports, database connections and replication lag; dashboards by route.
- **Backups:** production relies on managed PostgreSQL automated backups with point-in-time recovery and periodic restore tests; locally `pg_dump` of the Compose volume. Migration rollback: `pnpm db:rollback` reverts one migration; in production prefer roll-forward fixes with expand/contract migrations.
- **Connection pooling:** one pg pool per API process (max 10); with several replicas, PgBouncer (transaction mode) or the platform pooler caps total connections.

10. Trade-offs and roadmap

Deliberate omissions

- Trading, payments, advice and real market data are out of scope by design; prices come from the supplied snapshot.
- No sign-up, password reset, MFA or SSO: demo users only. Stateless JWT without refresh tokens or a revocation list.
- Admin import covers transactions only in the UI; the pipeline already supports every dataset.
- Transactions are an activity ledger and do not reconcile to the holdings snapshot (the supplied data does not); positions come from the snapshot.
- No cloud deployment, TLS or reverse-proxy certificates locally; documented production mapping instead.

Expected scale limits

- Offset pagination and `ILIKE` search are fine for thousands of rows; for millions: keyset pagination on `(trade_date, transaction_id)` and a `pg_trgm` index for search.
- Views compute valuations on read (sub-millisecond at this size, see Appendix B); at scale, a materialised daily valuation table refreshed after each snapshot load.
- CSV import holds the file in memory (5 MB cap); larger files would stream into `COPY` staging.

Next production steps

Priority	Risk	Next step
Performance	Transaction search and pagination degrade with ledger growth	Keyset pagination, trigram index, materialised valuations; measure with <code>EXPLAIN (ANALYZE, BUFFERS)</code> in CI on a large fixture
Security	Stateless session cannot be revoked before expiry	Short-lived access token + rotating refresh token, or server-side sessions; MFA/SSO for staff
Reliability	Single API instance and local database	Two+ replicas, managed PostgreSQL with PITR, alerting on import failures and 5xx
Data	Snapshot and ledger do not reconcile	Reconciliation job and exception queue for operations

Appendix A · Data-quality anomalies and handling

File · row	Anomaly	Handling
transactions · T0000026 (line 4552)	Duplicate ID, byte-identical to line 27	First kept, copy rejected <code>DUPLICATE_ROW</code> ; differing copies would reject all (<code>CONFLICTING_DUPLICATE</code>)
transactions · T0004551	Unknown instrument I9999	Rejected <code>UNKNOWN_REFERENCE</code>
transactions · T0004552	Negative FEE amount -75	Rejected <code>NEGATIVE_AMOUNT</code> ; not sign-flipped (could be a refund; guessing is unsafe)
transactions · T0004553	Trade dated 2027-01-05	Rejected <code>FUTURE_DATE</code>
holdings · 3 positions	Exact duplicate (account, instrument, date)	De-duplicated; PK prevents double-counted AUM
transactions · 1,069 rows	Trade before the account's open date	Imported, reported in <code>v_data_quality_exceptions</code> (rejecting would drop ~23% of the ledger)
transactions · 114 rows	Activity on CLOSED accounts	Imported, reported as exceptions; accounts shown with status badges
goals · most rows	Name mentions a different goal type	Soft flag <code>nameTypeMismatch</code>

All rejected rows are stored in `import_rejects` with every reason and the original row. Raw CSVs are committed unmodified with SHA-256 checksums.

Appendix B · SQL and performance evidence

All SQL tasks in the brief run against the views (queries and results in `docs/sql-tasks.md`): top 10 customers by AUM (C0026 ₹19,53,574.61), allocation per customer and for the whole book (equity 83.75%), monthly BUY/SELL net flow for 12 months, top instruments by distinct holders, 12 HIGH-priority goals under 25% funded, and data-quality exceptions. The book total ₹5,27,54,963.55 was cross-checked against an independent calculation from the raw CSVs.

EXPLAIN (ANALYZE, BUFFERS) for the transactions screen (one customer, date range, newest first, 25 rows):

```
Limit (actual time=0.056..0.059 rows=25) Buffers: shared hit=19
-> Sort Sort Key: trade_date DESC, transaction_id DESC (quicksort 29kB)
-> Nested Loop
    -> Seq Scan on accounts Filter: customer_id = 'C0026' (3 rows; 2 pages)
    -> Index Scan using transactions_account_date_idx on transactions
        Index Cond: account_id = accounts.account_id AND trade_date BETWEEN ...
Execution Time: 0.094 ms
```

The composite index jumps to each of the customer's accounts within the date range instead of scanning the 4,550-row ledger; 19 buffer hits, all from memory. The sequential scan on `accounts` is the planner's correct choice for a two-page table; `accounts_customer_id_idx` takes over as it grows.